

SplitBill AI

Definición del proyecto, análisis de requerimientos y diseño de la solución

Integrantes: [Completar nombres]

Asignatura: Lineamientos de desarrollo

Docente: Omar Andrés Cardona Zapata

Institución: [Completar institución]

Octubre de 2026

1. Definición del Proyecto

1.1. Título del proyecto

SplitBill AI - Divisor de Cuentas Inteligente.

SplitBill AI es una solución orientada a facilitar la división de gastos entre varias personas, permitiendo determinar de manera clara cuánto le corresponde pagar a cada participante de una cuenta. El concepto del producto busca reducir los cálculos manuales y evitar discusiones relacionadas con la distribución de los gastos. La propuesta contempla, como evolución del sistema, el reconocimiento de una factura mediante una fotografía para automatizar parte del proceso de cálculo (SplitBill AI, 2026, pp. 3-4).

1.2. Descripción general del problema a resolver

Cuando un grupo de personas comparte una comida, actividad o gasto común, no todos consumen o deben pagar necesariamente el mismo valor. Una persona puede realizar un consumo económico mientras otra adquiere productos de mayor valor; por ello, dividir el total de la cuenta en partes iguales puede resultar injusto. La presentación del producto señala además que los cálculos manuales pueden consumir tiempo y generar momentos incómodos o discusiones entre los participantes (SplitBill AI, 2026, p. 2).

SplitBill AI busca centralizar esta información y permitir que una cuenta pueda asociarse a diferentes usuarios, asignando a cada participante el monto que le corresponde. El sistema también permite identificar quién realizó inicialmente el pago, quién tiene valores pendientes y cuánto debe recibir el usuario que adelantó el dinero. Como evolución del producto, se plantea registrar consumos, capturar una fotografía de la factura y automatizar el cálculo mediante reconocimiento de la información contenida en el comprobante (SplitBill AI, 2026, pp. 3-4).

1.3. Público objetivo

SplitBill AI está dirigido principalmente a personas que realizan gastos compartidos y requieren distribuirlos de manera clara y organizada. La propuesta es aplicable a situaciones cotidianas como cenas, cumpleaños, salidas de trabajo, viajes grupales y actividades casuales de fin de semana (SplitBill AI, 2026, p. 6).

- Grupos de amigos que realizan comidas o salidas.
- Compañeros de trabajo que comparten gastos.
- Familias que desean distribuir consumos comunes.
- Personas que realizan viajes grupales.
- Participantes de celebraciones, cumpleaños o reuniones.
- Grupos en los que un integrante realiza inicialmente el pago por los demás.

1.4. Alcance del proyecto (qué incluye y qué no)

El alcance actual se concentra en la administración de usuarios, cuentas y divisiones, así como en la consulta de saldos y devoluciones. Las funciones de análisis de fotografías mediante inteligencia artificial se consideran una evolución futura de la solución y no forman parte de la API básica actualmente implementada.

Incluye	No incluye en la versión actual
Registro y administración de usuarios.	Reconocimiento automático de facturas mediante IA.
Creación y administración de cuentas.	Procesamiento OCR de imágenes.
Identificación del usuario pagador.	Aplicación móvil terminada.
Asignación de montos a participantes.	Integración con pasarelas bancarias reales.
Estados de pago PENDIENTE y PAGADO.	Procesamiento automático de impuestos y propinas desde una fotografía.

Incluye	No incluye en la versión actual
Cálculo de saldos, devoluciones y estado de distribución.	Notificaciones en tiempo real.
API REST documentada mediante Swagger/OpenAPI.	Autenticación avanzada o inicio de sesión social.

2. Análisis de Requerimientos

2.1. Requerimientos funcionales

ID	Requerimiento	Descripción
RF01	Gestión de usuarios	El sistema debe permitir crear, consultar, actualizar y eliminar usuarios.
RF02	Gestión de cuentas	El sistema debe permitir registrar una cuenta indicando descripción, monto total, fecha y usuario pagador.
RF03	División de cuentas	El sistema debe permitir distribuir una cuenta entre uno o varios usuarios mediante montos individuales.
RF04	Identificación del pagador	Cada cuenta debe estar asociada a un usuario responsable del pago inicial.
RF05	Consulta de participantes	El sistema debe permitir consultar los usuarios asociados a una determinada cuenta.
RF06	Consulta por usuario pagador	El sistema debe permitir consultar las cuentas pagadas inicialmente por un usuario.
RF07	Consulta de divisiones	El sistema debe permitir consultar divisiones por cuenta y por usuario.
RF08	Estado de pago	Cada división debe manejar los estados PENDIENTE y PAGADO.

ID	Requerimiento	Descripción
RF09	Información del pago	El sistema debe permitir almacenar tipo de pago, referencia y fecha de pago cuando aplique.
RF10	Cálculo de saldos	El sistema debe determinar cuánto debe pagar cada participante.
RF11	Cálculo de devolución	El sistema debe calcular cuánto debe recibir el usuario que adelantó el dinero.
RF12	Estado de distribución	El sistema debe determinar si la distribución está COMPLETA, PENDIENTE o EXCEDIDA.
RF13	Documentación de API	Los endpoints deben poder consultarse y probarse mediante Swagger/OpenAPI.
RF14	Registro de consumos - proyectado	En una evolución futura, los usuarios podrán registrar productos consumidos.
RF15	Lectura de factura - proyectado	En una evolución futura, la aplicación podrá capturar y analizar una fotografía de la factura.

2.2. Requerimientos no funcionales (deseables)

ID	Categoría	Descripción
RNF01	Rendimiento	Las operaciones habituales de consulta y registro deben responder de manera fluida.
RNF02	Seguridad	Las credenciales y contraseñas no deben exponerse en las respuestas de la API.
RNF03	Integridad	La base de datos debe conservar correctamente las relaciones entre usuarios, cuentas y divisiones.
RNF04	Usabilidad	La interfaz debe presentar de manera clara montos, participantes y estados de pago.

ID	Categoría	Descripción
RNF05	Mantenibilidad	La solución debe separar responsabilidades en capas para facilitar mantenimiento y crecimiento.
RNF06	Escalabilidad	La arquitectura debe permitir incorporar posteriormente OCR, IA y aplicaciones móviles.
RNF07	Documentación	La API debe contar con documentación actualizada mediante Swagger/OpenAPI.
RNF08	Compatibilidad	La API debe utilizar JSON para facilitar su consumo desde clientes web o móviles.
RNF09	Experiencia de usuario	El sistema debe reducir los cálculos manuales y minimizar errores durante la división de cuentas.

3. Diseño de la Solución

3.1. Arquitectura tecnológica general

La solución utiliza una arquitectura cliente-servidor. La aplicación cliente consume una API REST desarrollada en ASP.NET Core, y la API accede a SQL Server mediante Dapper. La organización interna separa modelos, consultas y repositorios para reducir el acoplamiento y facilitar el mantenimiento.



Componentes principales: Controllers - Query - Repository - Models - Dapper - Swagger/OpenAPI

3.2. Diagrama de flujo de navegación de la app

Paso	Acción	Continuidad
1	Registro o selección de usuario	↓

Paso	Acción	Continuidad
2	Crear cuenta: descripción, monto total, fecha y usuario pagador	↓
3	Agregar participantes	↓
4	Asignar valores individuales	↓
5	Consultar distribución y diferencia	↓
6	Consultar saldos: PAGA, RECIBE o SALDADO	↓
7	Actualizar pagos	↓
8	Cerrar la cuenta	FIN

La evolución proyectada del producto incorpora un flujo adicional compuesto por registro de pedidos, fotografía de la factura y cálculo automático. Este enfoque coincide con la propuesta de tres pasos presentada para SplitBill AI (SplitBill AI, 2026, p. 4).

3.3. Guías de estilo

La identidad visual de la aplicación toma como referencia la presentación de SplitBill AI. En ella predominan el azul oscuro, verde/turquesa, azul claro y fondos blancos o muy claros. La interfaz propuesta debe conservar una estética limpia, moderna y de alta legibilidad.

Elemento	Lineamiento
Paleta	Azul oscuro para títulos y elementos principales; verde/turquesa para acciones y énfasis; azul claro como apoyo; fondos blancos o muy claros.
Tipografía	Fuente sans serif de alta legibilidad en la aplicación. Títulos con mayor peso tipográfico y textos secundarios con peso regular.
Componentes	Tarjetas, botones claramente identificables, iconografía sencilla e indicadores visuales de estado.
Información monetaria	Los valores, saldos y estados deben distinguirse claramente y evitar ambigüedades.

Elemento	Lineamiento
Principios	Simplicidad, rapidez, transparencia, claridad y reducción de errores.
Estados	PAGA, RECIBE, SALDADO; y distribución COMPLETA, PENDIENTE o EXCEDIDA.

3.4. Modelo entidad-relación de la base de datos

El modelo de datos se compone de tres entidades principales: Usuario, Cuenta y Division. Division actúa como la relación entre una cuenta y sus participantes, almacenando el monto correspondiente y el estado del pago.

USUARIO	CUENTA	DIVISION
PK id_usuario	PK id_cuenta	PK id_division
nombre	FK id_usuario_pagador	FK id_cuenta
correo	descripcion	FK id_usuario
password_hash	monto_total	monto
telefono	fecha	estado
		tipo_pago
		referencia_pago
		fecha_pago

Relaciones: Usuario 1:N Cuenta | Cuenta 1:N Division | Usuario 1:N Division

3.5. Especificación de endpoints de la API

La API se documenta mediante Swagger/OpenAPI, lo que permite consultar y probar los endpoints desde el navegador. Como evidencia adicional se recomienda entregar una captura general de Swagger o el archivo de especificación OpenAPI generado por la aplicación.

Módulo	Método	Ruta	Propósito
Usuario	GET	/api/Usuario	Consultar usuarios.

Módulo	Método	Ruta	Propósito
Usuario	GET	/api/Usuario/{id}	Consultar un usuario por ID.
Usuario	POST	/api/Usuario	Crear un usuario.
Usuario	PUT	/api/Usuario/{id}	Actualizar un usuario.
Usuario	DELETE	/api/Usuario/{id}	Eliminar un usuario.
Usuario	GET	/api/Usuario/cuenta/{idCuenta}/participantes	Consultar participantes de una cuenta.
Cuenta	GET	/api/Cuenta	Consultar cuentas.
Cuenta	GET	/api/Cuenta/{id}	Consultar una cuenta por ID.
Cuenta	POST	/api/Cuenta	Crear una cuenta.
Cuenta	PUT	/api/Cuenta/{id}	Actualizar una cuenta.
Cuenta	DELETE	/api/Cuenta/{id}	Eliminar una cuenta.
Cuenta	GET	/api/Cuenta/usuario/{idUser}	Consultar cuentas por usuario pagador.
Cuenta	GET	/api/Cuenta/{idCuenta}/devoluciones	Consultar saldos, devoluciones y estado de distribución.
Division	GET	/api/Division	Consultar divisiones.
Division	GET	/api/Division/{id}	Consultar una división por ID.
Division	POST	/api/Division	Crear una división.
Division	PUT	/api/Division/{id}	Actualizar una división.
Division	DELETE	/api/Division/{id}	Eliminar una división.
Division	GET	/api/Division/cuenta/{idCuenta}	Consultar divisiones de una cuenta.
Division	GET	/api/Division/usuario/{idUser}	Consultar divisiones de un usuario.
Diagnóstico	GET	/dbtest	Comprobar conexión y tablas durante desarrollo.

Referencias

Cardona Zapata, O. A. (s. f.). Lineamientos de desarrollo [Documento de instrucciones].

SplitBill AI. (2026). SplitBill AI - Presentación de Producto [Presentación de proyecto].

Nota. Las funciones de reconocimiento de facturas mediante fotografía e inteligencia artificial se presentan como evolución del producto en el material de referencia; la API descrita en este documento corresponde a la implementación actual basada en Usuario, Cuenta y Division.